

LAPORAN PRAKTIKUM MODUL 10: SHELL



**Universitas
Telkom**

Disusun Oleh :

Nama : Ganes Gemi Putra

Kelas : SE-07-02

NIM : (2311104075)

Dosen : Yudha Islami Sulistya S.Kom M.Cs

**PROGRAM STUDI S1 SOFTWARE ENGINEERING
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY
PURWOKERTO
2026**



DAFTAR ISI

BAB I PENDAHULUAN
BAB II LANDASAN TEORI
BAB III PERANCANGAN DAN KEBUTUHAN
BAB IV IMPLEMENTASI
BAB V PENGUJIAN DAN ANALISIS HASIL
BAB VI PENUTUP
LAMPIRAN SOURCE CODE DAN DOKUMENTASI

BAB I

PENDAHULUAN

1.1 Latar Belakang

Shell pada sistem operasi berfungsi sebagai antarmuka antara pengguna dan kernel. Melalui shell, pengguna dapat menjalankan perintah, melihat informasi proses, serta memanggil fungsi sistem yang sudah disediakan oleh kernel. Pada praktikum Modul 10, dilakukan modifikasi terhadap Xinu dengan menambahkan syscall chname dan command baru bernama namecmd.

Syscall chname digunakan untuk mengubah nama proses berdasarkan PID. Setelah syscall berhasil dibuat dan didaftarkan, command namecmd digunakan untuk memanggil syscall tersebut dari shell Xinu. Pengujian dilakukan dengan membandingkan hasil perintah ps sebelum dan sesudah command dijalankan.

1.2 Rumusan Masalah

- Bagaimana cara mengubah prototype syscall chname agar menerima parameter PID, nama baru, dan panjang nama?
- Bagaimana cara memodifikasi chname.c agar nama proses dapat diganti melalui proses penyalinan karakter?
- Bagaimana cara menambahkan command shell baru bernama namecmd?
- Bagaimana membuktikan bahwa nama proses berubah setelah command namecmd dijalankan?

1.3 Tujuan Praktikum

- Mahasiswa mampu memodifikasi shell pada Xinu.
- Mahasiswa mampu menambahkan dan memodifikasi syscall pada Xinu.
- Mahasiswa mampu membuat command baru pada shell Xinu.
- Mahasiswa mampu melakukan proses compile dan pengujian hasil modifikasi melalui terminal Xinu.

1.4 Identitas Praktikum

Nama	Ganes Gemi Putra
NIM	2311104075
Kelas	SE-07-02
Mata Praktikum	Sistem Operasi
Modul	Modul 10 - Shell
Dosen	Yudha Islami Sulistya S.Kom M.Cs



BAB II

LANDASAN TEORI

2.1 Shell pada Xinu

Shell merupakan program yang membaca input pengguna, mencocokkan nama perintah dengan daftar command yang tersedia, lalu menjalankan fungsi yang sesuai.

Pada Xinu, command shell didaftarkan pada tabel command sehingga shell dapat mengenali nama perintah baru.

2.2 Syscall

System call atau syscall adalah mekanisme untuk mengakses layanan kernel. Pada praktikum ini, syscall chname dimodifikasi agar dapat mengganti nama proses yang ada pada tabel proses Xinu.

Prototype fungsi perlu disesuaikan agar pemanggilan fungsi sesuai dengan parameter yang dibutuhkan.

Process Table dan Nama Proses

Setiap proses pada Xinu memiliki entri pada process table. Salah satu atribut pada entri proses adalah prname yang menyimpan nama proses. Untuk mengganti nama proses, program perlu mengambil alamat entri proses berdasarkan PID, kemudian menyalin karakter nama baru ke prname.

2.4 Fungsi namecmd

namecmd adalah command baru yang dibuat pada praktikum ini. Command tersebut menampilkan pesan "My new command" dan memanggil syscall chname untuk mengganti nama proses netin menjadi hello.

BAB III

PERANCANGAN DAN KEBUTUHAN

3.1 Perangkat dan File yang Digunakan

Sistem	Xinu Development VM pada Oracle VM VirtualBox
Editor	gedit
Terminal	Linux Terminal dan Minicom
File prototype	include/prototypes.h
File syscall	system/chname.c
File command baru	shell/xsh_namecmd.c
File tabel command	shell/shell.c
File tambahan yang disesuaikan	shell/xsh_uptime.c

3.2 Rancangan Alur Program

- Pengguna menjalankan perintah ps untuk melihat daftar proses awal.
- Pengguna menjalankan command namecmd pada shell Xinu.
- Command namecmd memanggil chname dengan parameter PID, string nama baru, dan panjang string.
- Fungsi chname memvalidasi PID, mengambil pointer process table, lalu menyalin karakter nama baru ke pname.
- Pengguna menjalankan ps lagi untuk membuktikan nama proses berubah.

3.3 Penyesuaian PID pada Lingkungan Praktikum

Pada instruksi soal, contoh pemanggilan menggunakan PID 3. Akan tetapi, pada hasil perintah ps di VM praktikum ini, proses netin berada pada PID 2. Oleh karena itu, pemanggilan chname pada file xsh_namecmd.c disesuaikan menjadi chname(2, "hello", 5) agar proses netin yang benar-benar muncul pada sistem dapat berubah menjadi hello.

BAB IV IMPLEMENTASI

4.1 Modifikasi prototypes.h

Prototype chname diubah agar memiliki tiga parameter, yaitu pid32 sebagai PID proses, char * sebagai nama baru, dan uint32 sebagai panjang nama. Selain itu, prototype xsh_namecmd juga ditambahkan agar command baru dapat dikenali saat kompilasi.

Potongan kode prototypes.h:

```
/* in file xsh_namecmd.c */
extern shellcmd xsh_namecmd(int, char *[]);

/* in file chname.c */
extern pri16 chname(pid32, char *, uint32);
```



```
Activities gedit Mon 4 May, 22:01
Open prototypes.h Save
~/xinu/include

extern void arp_in(struct arppacket *);
extern int32 arp_alloc(void);
extern void arp_ntoh(struct arppacket *);
extern void arp_hton(struct arppacket *);

/* in file ascdatetime.c */
extern status ascdatetime(uint32, char *);

/* in file bufinit.c */
extern status bufinit(void);

/* in file chprio.c */
extern syscall chprio(pid32, pri16);

/* in file xsh_namecmd.c */
extern shellcmd xsh_namecmd(int, char *[]);

/* in file chname.c */
extern pri16 chname(pid32, char *, uint32);

/* in file clkupdate.S */
extern uint32 clkcount(void);

/* in file clkhandler.c */
extern interrupt clkhandler(void);

/* in file clkinit.c */
extern void clkinit(void);

C/C++/ObjC Header Tab Width: 8 Ln 30, Col 23 INS
```

Gambar 1. Prototype chname dan xsh_namecmd pada include/prototypes.h.

4.2 Modifikasi chname.c

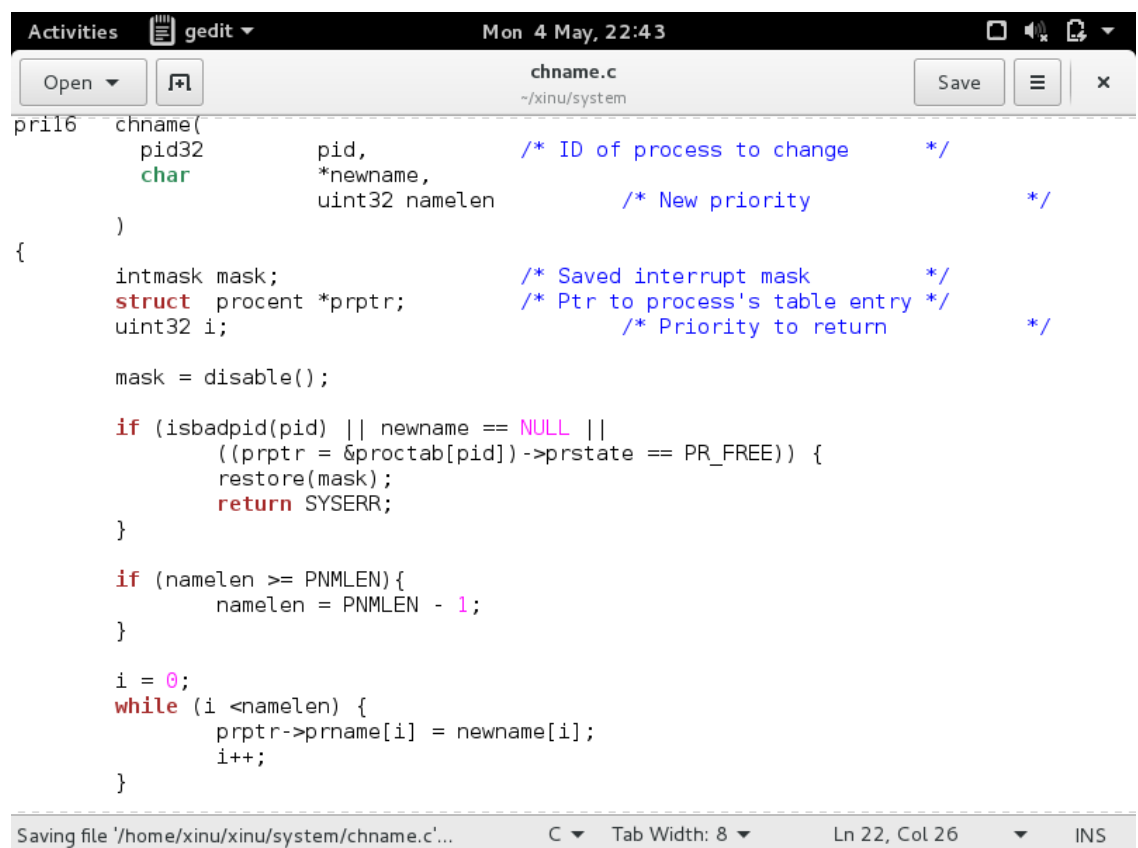
Pada file chname.c, bagian yang sebelumnya berkaitan dengan newprio dihapus. Fungsi kemudian diganti agar menerima parameter newname dan namelen. Selama nilai i lebih kecil dari namelen, karakter dari newname[i] disalin ke prptr->prname[i].

Potongan kode utama chname.c:

```
if (isbadpid(pid) || newname == NULL ||
    ((prptr = &proctab[pid])->prstate == PR_FREE)) {
    restore(mask);
    return SYSERR;
}

if (namelen >= PNMLEN) {
    namelen = PNMLEN - 1;
}

i = 0;
while (i < namelen) {
    prptr->prname[i] = newname[i];
    i++;
}
prptr->prname[i] = '\0';
```



```
Activities gedit Mon 4 May, 22:43
chname.c ~/xinu/system
Open Save

pril6 chname(
    pid32 pid, /* ID of process to change */
    char *newname,
    uint32 namelen /* New priority */
)
{
    intmask mask; /* Saved interrupt mask */
    struct proctent *prptr; /* Ptr to process's table entry */
    uint32 i; /* Priority to return */

    mask = disable();

    if (isbadpid(pid) || newname == NULL ||
        ((prptr = &proctab[pid])->prstate == PR_FREE)) {
        restore(mask);
        return SYSERR;
    }

    if (namelen >= PNMLEN) {
        namelen = PNMLEN - 1;
    }

    i = 0;
    while (i < namelen) {
        prptr->prname[i] = newname[i];
        i++;
    }
}
```

Saving file '/home/xinu/xinu/system/chname.c'... C Tab Width: 8 Ln 22, Col 26 INS

Gambar 2. Implementasi fungsi chname pada file system/chname.c.

4.3 Pembuatan Command xsh_namecmd.c

Command baru dibuat pada file shell/xsh_namecmd.c. Command ini mencetak pesan sebagai bukti command berhasil dijalankan, kemudian memanggil syscall chname untuk mengganti nama proses netin menjadi hello.

Pada proses awal, include stdio.h dan string.h sempat ditambahkan sesuai arahan praktikum. Namun, pada lingkungan compiler Xinu yang digunakan, header stdio.h tidak tersedia sehingga final code disesuaikan dengan menggunakan include xinu.h yang sudah menyediakan fungsi yang dibutuhkan.

Source code final xsh_namecmd.c:

```
/* Nama: Ganes Gemi Putra
 * NIM : 2311104075
 */

#include <xinu.h>

shellcmd xsh_namecmd(int nargs, char *args[])
{
    printf("My new command\n");

    /* pid 2 = netin; mengubah nama proses pid 2 menjadi hello */
    chname(2, "hello", 5);

    return 0;
}
```



```
Activities gedit Mon 4 May, 22:59
xsh_namecmd.c ~/xinu/shell
Save

/* Nama: Ganes Gemi Putra
 * NIM : 2311104075
 */

#include <xinu.h>

shellcmd xsh_namecmd(int nargs, char *args[])
{
    printf("My new command\n");

    /* pid 2 = netin; mengubah nama proses pid 2 menjadi hello */
    chname(2, "hello", 5);

    return 0;
}

Saving file '/home/xinu/xinu/shell/xsh_namecmd.c'... C Tab Width: 8 Ln 12, Col 28 INS
```

Gambar 3. Source code final command namecmd pada file shell/xsh_namecmd.c.

4.4 Pendaftaran Command pada shell.c

Agar command namecmd dapat dipanggil dari shell, command tersebut ditambahkan ke tabel cmdtab pada file shell.c. Struktur tabel pada versi Xinu ini menggunakan dua kolom, yaitu nama command dan fungsi

command.

Potongan kode shell.c:

```
{ "namecmd", xsh_namecmd},
```



```

Activities  gedit  Mon 4 May, 21:52
Open  shell.c  Save
~/xinu/shell

{"cat", xsh_cat},
{"clear", xsh_clear},
{"date", xsh_date},
{"devdump", xsh_devdump},
{"echo", xsh_echo},
{"help", xsh_help},
{"ls", xsh_ls},
{"kill", xsh_kill},
{"memdump", xsh_memdump},
{"memstat", xsh_memstat},
{"ns", xsh_ns},
{"netinfo", xsh_netinfo},
{"ping", xsh_ping},
{"namecmd", xsh_namecmd},
{"ps", xsh_ps},
{"sleep", xsh_sleep},
{"tee", xsh_tee},
{"udp", xsh_udpdump},
{"udpecho", xsh_udpecho},
{"udpserver", xsh_udpserver},
{"uptime", xsh_uptime},
{"?", xsh_help}

};

uint32 ncmd = sizeof(cmdtab) / sizeof(struct cmdent);
  
```

Gambar 4. Penambahan command namecmd pada tabel command shell.c.

4.5 Penyesuaian File xsh_uptime.c

Saat proses compile, ditemukan bahwa file xsh_uptime.c masih memanggil chname dengan format lama, yaitu chname(5,33). Karena prototype chname sudah berubah menjadi tiga parameter, pemanggilan lama tersebut menyebabkan error. Baris tersebut kemudian dikomentari agar tidak mengganggu proses kompilasi.

Perbaikan pada xsh_uptime.c:

```
// chname(5,33);
```



```
Activities gedit Mon 4 May, 22:54
xsh_uptime.c
~/xinu/shell

hrs -= secs/secperhr;
secs -= hrs*secperhr;

/* Subtract number of minutes */

mins = secs/secpermin;
secs -= mins*secpermin;

printf("Xinu has been up ");
if (days > 0) {
    printf(" %d day(s) ", days);
}

if (hrs > 0) {
    printf(" %d hour(s) ", hrs);
}

if (mins > 0) {
    printf(" %d minute(s) ", mins);
}

if (secs > 0) {
    printf(" %d second(s) ", secs);
}
printf("\n");
//chname(5,33);
return 0;
}
```

C Tab Width: 8 Ln 75, Col 11 INS

Gambar 5. Baris chname lama pada xsh_uptime.c dikomentari agar compile berhasil.

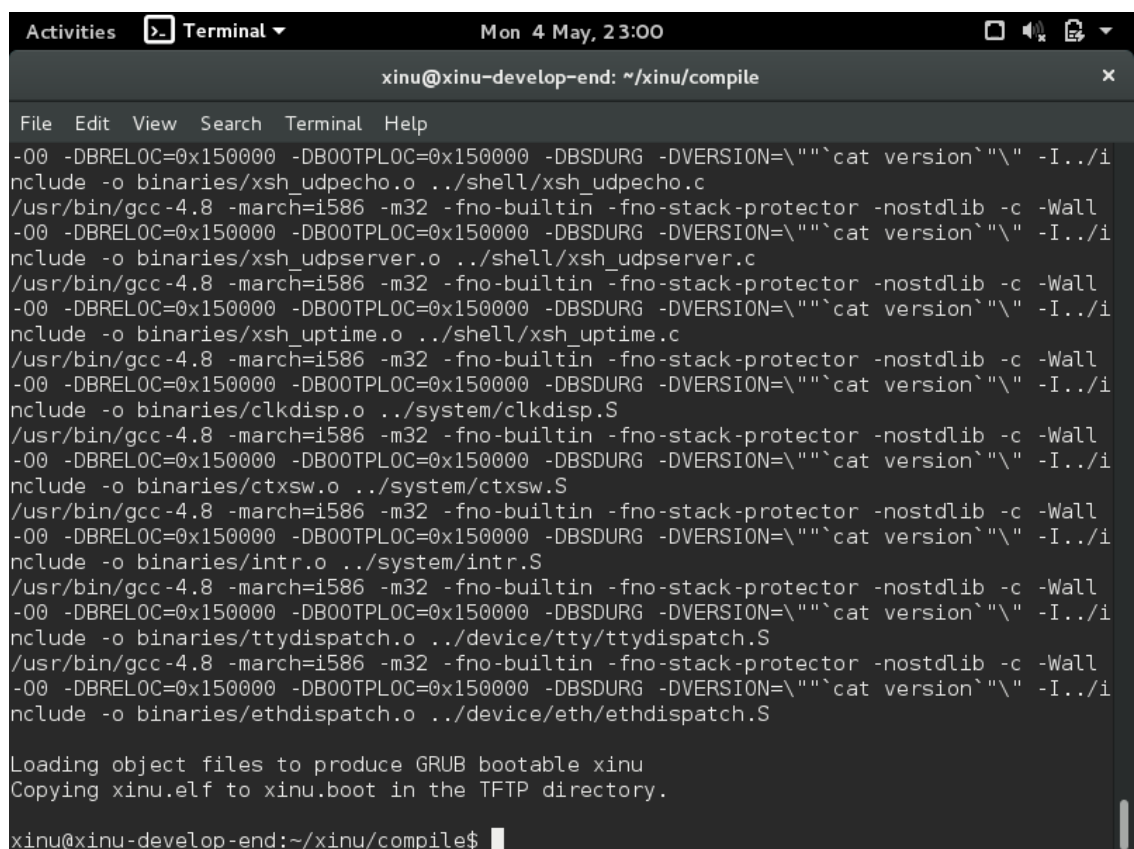
BAB V PENGUJIAN DAN ANALISIS HASIL

5.1 Proses Compile

Setelah seluruh perubahan disimpan, sistem dikompilasi ulang melalui direktori compile menggunakan perintah `make clean` dan `make`. Kompilasi dinyatakan berhasil ketika muncul proses pembuatan GRUB bootable Xinu dan penyalinan `xinu.elf` ke `xinu.boot` pada direktori TFTP.

Perintah compile:

```
cd /home/xinu/xinu/compile
make clean
make
```



```

xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../i
nclude -o binaries/clkdsp.o ../system/clkdsp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLC=0x150000 -DBSDURG -DVERSION=\"`cat version`\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$

```

Gambar 6. Hasil compile berhasil tanpa error.

5.2 Pengujian pada Terminal Xinu

Pengujian dilakukan melalui terminal Xinu menggunakan `minicom`. Perintah `ps` dijalankan sebelum command `namecmd` untuk melihat kondisi awal. Setelah `namecmd` dijalankan, `ps` dijalankan kembali untuk membuktikan perubahan nama proses.

Urutan pengujian:

```
ps
namecmd
ps
```

```

Activities Terminal Mon 4 May, 23:02
xinu@xinu-develop-end: ~/xinu/compile

Welcome to Xinu!

xsh $ ps
Pid Name      State Prio Ppid Stack Base Stack Ptr  Stack Size Msg Content
-----
0 prnull      ready  0    0  0x005FDFFC 0x00FFFE4  8192
1 ipout       wait   500  0  0x005FBFFC 0x005FBF30  8192
2 netin       wait   500  0  0x005F9FFC 0x005F9ED0  8192
5 shell       recv   50   4  0x005D7FFC 0x005D79AC  8192
6 ps          curr   20   5  0x005F7FFC 0x005F7FC4  8192

xsh $ namecmd
My new command
xsh $ ps
Pid Name      State Prio Ppid Stack Base Stack Ptr  Stack Size Msg Content
-----
0 prnull      ready  0    0  0x005FDFFC 0x00FFFE4  8192
1 ipout       wait   500  0  0x005FBFFC 0x005FBF30  8192
2 hello       wait   500  0  0x005F9FFC 0x005F9ED0  8192
5 shell       recv   50   4  0x005D7FFC 0x005D79AC  8192
8 ps          curr   20   5  0x005F7FFC 0x005F7FC4  8192

xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
  
```

Gambar 7. Hasil akhir: proses netin pada PID 2 berubah menjadi hello setelah namecmd dijalankan.

5.3 Analisis Hasil

Pada hasil ps pertama, proses dengan PID 2 memiliki nama netin. Setelah command namecmd dijalankan, shell menampilkan pesan "My new command" sebagai tanda command berhasil dipanggil. Pada hasil ps kedua, proses PID 2 berubah nama menjadi hello. Hal ini membuktikan bahwa syscall chname berhasil memodifikasi field pname pada process table.

Perubahan nama proses terjadi karena fungsi chname menyalin karakter dari string newname ke array pname milik proses target. Penambahan karakter null terminator pada akhir string juga penting agar nama proses terbaca dengan benar oleh perintah ps.



BAB VI

PENUTUP

6.1 Kesimpulan

- Syscall chname berhasil dimodifikasi sehingga dapat menerima parameter PID, nama baru, dan panjang nama.
- Fungsi chname berhasil mengganti nama proses dengan cara menyalin karakter dari newname ke prptr->prname.
- Command namecmd berhasil ditambahkan ke shell Xinu dan dapat dijalankan dari prompt xsh.
- Pengujian berhasil menunjukkan perubahan nama proses dari netin menjadi hello.

6.2 Saran

Pada praktikum selanjutnya, sebaiknya setiap perubahan prototype syscall langsung diperiksa pada seluruh file yang memanggil fungsi tersebut. Hal ini dapat mencegah error compile akibat masih adanya pemanggilan fungsi dengan format lama.

LAMPIRAN SOURCE CODE FINAL

Lampiran 1 - include/prototypes.h:

```
/* in file xsh_namecmd.c */
extern shellcmd xsh_namecmd(int, char *[]);

/* in file chname.c */
extern pri16 chname(pid32, char *, uint32);
```

Lampiran 2 - system/chname.c:

```
#include <xinu.h>

pri16 chname(pid32 pid, char *newname, uint32 namelen)
{
    intmask mask;
    struct procent *prptr;
    uint32 i;

    mask = disable();

    if (isbadpid(pid) || newname == NULL ||
        ((prptr = &proctab[pid])->prstate == PR_FREE)) {
        restore(mask);
        return SYSERR;
    }

    if (namelen >= PNMLEN) {
        namelen = PNMLEN - 1;
    }

    i = 0;
    while (i < namelen) {
        prptr->prname[i] = newname[i];
        i++;
    }

    prptr->prname[i] = '\0';

    restore(mask);
    return OK;
}
```

Lampiran 3 - shell/xsh_namecmd.c:

```
/* Nama: Ganes Gemi Putra
 * NIM : 2311104075
 */

#include <xinu.h>

shellcmd xsh_namecmd(int nargs, char *args[])
{
    printf("My new command\n");
    chname(2, "hello", 5);
    return 0;
}
```

Lampiran 4 - shell/shell.c:

```
{"namecmd", xsh_namecmd},
```